

Python NumPy Cheat Sheet

A practical quick reference for beginners & intermediate users
Ordered from the most commonly used functions to the least

EVERY EXAMPLE IN THIS GUIDE USES THESE ARRAYS

```
import numpy as np

# Values taken from HR_ListOfEmployees.csv
salary = np.array([69500, 91500, 100000, 102000,
                  106500, 62500, 72000, 111000])
age     = np.array([40, 37, 36, 41, 24, 59, 45, 38])
rating  = np.array([3.6, 4.6, 3.0, 3.0, 3.1, 3.4, 2.6, 4.6])
```

FOUR IDEAS THAT EXPLAIN MOST OF NUMPY

ndarray	The core NumPy object: a fast, fixed-type grid of numbers.
Vectorised	Maths applies to every item at once - no Python loops needed.
dtype	Every array has one data type, e.g. int64 or float64.
Axis	axis=0 works down columns, axis=1 works across rows.

PREVIEW — THE salary ARRAY

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
69500	91500	100000	102000	106500	62500	72000	111000

Blue bar = code you type

Green bar = real output returned

148 examples · **14** topics

Every sample output was produced by actually running the code on NumPy 2.3. Companion guide to the Pandas cheat sheet.

Table of Contents

01	Creating Arrays Turn lists and ranges into arrays.	12 functions	3
02	Array Info & Inspection Check size, shape and data type.	10 functions	5
03	Indexing & Slicing Grab single items or a range.	12 functions	6
04	Filtering with Conditions Keep only the values you want.	12 functions	7
05	Math & Arithmetic Do maths on whole arrays at once.	12 functions	8
06	Statistics & Aggregation Summarise numbers in one step.	16 functions	9
07	Sorting & Searching Put values in order and find them.	8 functions	11
08	Reshaping & Transposing Change the shape of your data.	10 functions	12
09	Combining & Splitting Join arrays or break them apart.	11 functions	13
10	Copying & Modifying Change arrays without breaking originals.	8 functions	14
11	Handling Missing Values Work with NaN (empty) numbers.	9 functions	15
12	Random Numbers Make sample data and random picks.	8 functions	16
13	Linear Algebra Matrix maths used in data science.	8 functions	17
14	Useful Utilities Small helpers that save time.	12 functions	18

1. Creating Arrays

12 functions

Turn lists and ranges into arrays.

1.1 `import numpy as np` # Standard way to import NumPy

SAMPLE OUTPUT

```
2.3.5
```

1.2 `salary = np.array([69500, 91500, 100000, 102000])` # Creates an array from a list

SAMPLE OUTPUT

```
[ 69500  91500 100000 102000]
```

1.3 `np.array([[69500, 40], [91500, 37]])` # Creates a 2D array (rows and columns)

SAMPLE OUTPUT

```
[[69500  40]
 [91500  37]]
```

1.4 `np.zeros(5)` # Array filled with zeros

SAMPLE OUTPUT

```
[0. 0. 0. 0. 0.]
```

1.5 `np.ones(4)` # Array filled with ones

SAMPLE OUTPUT

```
[1. 1. 1. 1.]
```

1.6 `np.arange(0, 10, 2)` # Numbers in a range with a step

SAMPLE OUTPUT

```
[0 2 4 6 8]
```

1.7 `np.linspace(0, 1, 5)` # Evenly spaced numbers between two values

SAMPLE OUTPUT

```
[0.  0.25 0.5  0.75 1.  ]
```

1.8 `np.full(4, 50000)` # Array filled with one value

SAMPLE OUTPUT

```
[50000 50000 50000 50000]
```

1.9 `np.zeros((2, 3))` # Grid of zeros with a shape

SAMPLE OUTPUT

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

1.10 `np.eye(3)` # Identity matrix with ones diagonally

SAMPLE OUTPUT

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

1.11 `np.empty(3).shape` # Fast uninitialised array

SAMPLE OUTPUT

```
(3)
```

1.12 `np.array([1, 2, 3], dtype="float64")` # Creates an array with a set type

SAMPLE OUTPUT

```
[1. 2. 3.]
```

2. Array Info & Inspection

10 functions

Check size, shape and data type.

2.1 `salary.shape` # Shows rows and columns

SAMPLE OUTPUT

```
(8)
```

2.2 `salary.size` # Counts all the items

SAMPLE OUTPUT

```
8
```

2.3 `salary.ndim` # Number of dimensions

SAMPLE OUTPUT

```
1
```

2.4 `salary.dtype` # Shows the data type

SAMPLE OUTPUT

```
dtype('int64')
```

2.5 `len(salary)` # Length of the first dimension

SAMPLE OUTPUT

```
8
```

2.6 `salary.astype("float64")` # Changes the data type

SAMPLE OUTPUT

```
[ 69500.  91500. 100000. 102000. 106500.  62500.  72000. 111000.]
```

2.7 `grid.shape` # Shape of a 2D array

SAMPLE OUTPUT

```
(3, 2)
```

2.8 `salary.itemsize` # Bytes used by one item

SAMPLE OUTPUT

```
8
```

2.9 `salary.nbytes` # Total bytes used by the array

SAMPLE OUTPUT

```
64
```

2.10 `salary.tolist()` # Converts the array back to a list

SAMPLE OUTPUT

```
[69500, 91500, 100000, 102000, 106500, 62500, 72000, 111000]
```

3. Indexing & Slicing

12 functions

Grab single items or a range.

3.1 `salary[0]` # Gets the first item

SAMPLE OUTPUT

```
69500
```

3.2 `salary[-1]` # Gets the last item

SAMPLE OUTPUT

```
111000
```

3.3 `salary[0:3]` # Gets a slice of items

SAMPLE OUTPUT

```
[ 69500  91500 100000]
```

3.4 `salary[:4]` # Gets everything up to an index

SAMPLE OUTPUT

```
[ 69500  91500 100000 102000]
```

3.5 `salary[4:]` # Gets everything from an index

SAMPLE OUTPUT

```
[106500  62500  72000 111000]
```

3.6 `salary[::2]` # Takes every second item

SAMPLE OUTPUT

```
[ 69500 100000 106500  72000]
```

3.7 `salary[::-1]` # Reverses the array

SAMPLE OUTPUT

```
[111000  72000  62500 106500 102000 100000  91500  69500]
```

3.8 `salary[[0, 2, 5]]` # Picks items at chosen positions

SAMPLE OUTPUT

```
[ 69500 100000  62500]
```

3.9 `grid[1, 0]` # One cell from a 2D array

SAMPLE OUTPUT

```
91500
```

3.10 `grid[:, 0]` # Gets a whole column

SAMPLE OUTPUT

```
[ 69500  91500 100000]
```

3.11 `grid[0, :]` # Gets a whole row

SAMPLE OUTPUT

```
[69500  40]
```

3.12 `salary[2] = 95000` # Changes a value in place

SAMPLE OUTPUT

```
[ 69500  91500  95000 102000 106500  62500  72000 111000]
```

4. Filtering with Conditions

12 functions

Keep only the values you want.

4.1 `salary > 90000` # Returns True or False per item

SAMPLE OUTPUT

```
[False True True True True False False True]
```

4.2 `salary[salary > 90000]` # Keeps only values above a number

SAMPLE OUTPUT

```
[ 91500 100000 102000 106500 111000]
```

4.3 `age[age < 40]` # Keeps values below a number

SAMPLE OUTPUT

```
[37 36 24 38]
```

4.4 `salary[(salary > 70000) & (salary < 105000)]` # Two conditions, both must be true

SAMPLE OUTPUT

```
[ 91500 100000 102000 72000]
```

4.5 `age[(age < 30) | (age > 55)]` # Two conditions, either can be true

SAMPLE OUTPUT

```
[24 59]
```

4.6 `salary[~(salary > 90000)]` # Keeps values that do NOT match

SAMPLE OUTPUT

```
[69500 62500 72000]
```

4.7 `np.where(salary > 90000, "High", "Low")` # Labels values by a condition

SAMPLE OUTPUT

```
['Low' 'High' 'High' 'High' 'High' 'Low' 'Low' 'High']
```

4.8 `np.where(salary > 90000)[0]` # Positions where the rule is true

SAMPLE OUTPUT

```
[1 2 3 4 7]
```

4.9 `(salary > 90000).sum()` # Counts how many match

SAMPLE OUTPUT

```
5
```

4.10 `(salary > 90000).any()` # Checks if any value matches

SAMPLE OUTPUT

```
True
```

4.11 `(salary > 50000).all()` # Checks if all values match

SAMPLE OUTPUT

```
True
```

4.12 `np.isin(age, [36, 40, 59])` # Checks membership in a list

SAMPLE OUTPUT

```
[ True False  True False False  True False False]
```

5. Math & Arithmetic

12 functions

Do maths on whole arrays at once.

5.1 `salary + 5000` # Adds a number to every item

SAMPLE OUTPUT

```
[ 74500  96500 105000 107000 111500  67500  77000 116000]
```

5.2 `salary * 1.10` # Gives everyone a 10 percent raise

SAMPLE OUTPUT

```
[ 76450. 100650. 110000. 112200. 117150.  68750.  79200. 122100.]
```

5.3 `salary / 12` # Divides every item

SAMPLE OUTPUT

```
[5791.67 7625.  8333.33 8500.  8875.  5208.33 6000.  9250. ]
```

5.4 `salary - age * 100` # Combines two arrays in one formula

SAMPLE OUTPUT

```
[ 65500  87800  96400  97900 104100  56600  67500 107200]
```

5.5 `rating * 2` # Multiplies each item

SAMPLE OUTPUT

```
[7.2 9.2 6.  6.  6.2 6.8 5.2 9.2]
```

5.6 `salary ** 0.5` # Raises items to a power

SAMPLE OUTPUT

```
[263.6 302.5 316.2 319.4 326.3 250.  268.3 333.2]
```

5.7 `np.round(salary / 12, 2)` # Rounds to a set number of decimals

SAMPLE OUTPUT

```
[5791.67 7625.  8333.33 8500.  8875.  5208.33 6000.  9250. ]
```

5.8 `np.sqrt(salary).round(1)` # Square root of each item

SAMPLE OUTPUT

```
[263.6 302.5 316.2 319.4 326.3 250.  268.3 333.2]
```

5.9 `np.abs(np.array([-5, 3, -8]))` # Removes the minus sign

SAMPLE OUTPUT

```
[5 3 8]
```

5.10 `salary // 1000` # Whole-number division

SAMPLE OUTPUT

```
[ 69  91 100 102 106  62  72 111]
```

5.11 `salary % 1000` # Remainder after division

SAMPLE OUTPUT

```
[500 500  0  0 500 500  0  0]
```

5.12 `np.log(salary).round(2)` # Natural logarithm of each item

SAMPLE OUTPUT

```
[11.15 11.42 11.51 11.53 11.58 11.04 11.18 11.62]
```

6. Statistics & Aggregation

16 functions

Summarise numbers in one step.

6.1 `salary.mean()` # Calculates the average

SAMPLE OUTPUT

```
89375.0
```

6.2 `salary.sum()` # Adds up all values

SAMPLE OUTPUT

```
715000
```

6.3 `salary.min()` # Finds the smallest value

SAMPLE OUTPUT

```
62500
```

6.4 `salary.max()` # Finds the largest value

SAMPLE OUTPUT

```
111000
```

6.5 `np.median(salary)` # Finds the middle value

SAMPLE OUTPUT

```
95750.0
```

6.6 `salary.std().round(2)` # Measures how spread out values are

SAMPLE OUTPUT

```
17524.54
```

6.7 `salary.var().round(2)` # Variance of the values

SAMPLE OUTPUT

```
307109375.0
```

6.8 `np.percentile(salary, 75)` # Value below which 75 percent fall

SAMPLE OUTPUT

```
103125.0
```

6.9 `salary.argmax()` # Position of the largest value

SAMPLE OUTPUT

```
7
```

6.10 `salary.argmin()` # Position of the smallest value

SAMPLE OUTPUT

```
5
```

6.11 `np.unique(np.array(["IT", "Sales", "IT"]))` # Unique values, sorted

SAMPLE OUTPUT

```
['IT' 'Sales']
```

6.12 `np.unique(dept, return_counts=True)` # Unique values with their counts

SAMPLE OUTPUT

```
(array(['HR', 'IT', 'Sales']), array([1, 3, 1]))
```

6.13 `salary.cumsum()` # Running total down the array

SAMPLE OUTPUT

```
[ 69500 161000 261000 363000 469500 532000 604000 715000]
```

6.14 `np.corrcoef(salary, age)[0, 1].round(3)` # How strongly two arrays relate

SAMPLE OUTPUT

```
-0.74
```

6.15 `grid.sum(axis=0)` # Totals down each column

SAMPLE OUTPUT

```
[261000  113]
```

6.16 `grid.mean(axis=1)` # Averages across each row

SAMPLE OUTPUT

```
[34770.  45768.5 50018. ]
```

7. Sorting & Searching

8 functions

Put values in order and find them.

7.1 `np.sort(salary)` # Sorts values smallest first

SAMPLE OUTPUT

```
[ 62500  69500  72000  91500 100000 102000 106500 111000]
```

7.2 `np.sort(salary)[::-1]` # Sorts values largest first

SAMPLE OUTPUT

```
[111000 106500 102000 100000  91500  72000  69500  62500]
```

7.3 `np.argsort(salary)` # Positions that would sort the array

SAMPLE OUTPUT

```
[5 0 6 1 2 3 4 7]
```

7.4 `np.sort(salary)[-3:]` # Gets the three largest values

SAMPLE OUTPUT

```
[102000 106500 111000]
```

7.5 `np.searchsorted(np.sort(salary), 95000)` # Where a value would be inserted

SAMPLE OUTPUT

```
4
```

7.6 `np.argwhere(salary > 100000).flatten()` # All positions matching a rule

SAMPLE OUTPUT

```
[3 4 7]
```

7.7 `np.sort(names)` # Sorts text alphabetically

SAMPLE OUTPUT

```
['Alvin Luna' 'Alvin Ramos' 'Amalia Alonzo']
```

7.8 `np.flip(age)` # Reverses the order of items

SAMPLE OUTPUT

```
[38 45 59 24 41 36 37 40]
```

8. Reshaping & Transposing

10 functions

Change the shape of your data.

8.1 `salary.reshape(2, 4)` # Reshapes into rows and columns

SAMPLE OUTPUT

```
[[ 69500  91500 100000 102000]
 [106500  62500  72000 111000]]
```

8.2 `salary.reshape(4, 2)` # Reshapes into a different grid

SAMPLE OUTPUT

```
[[ 69500  91500]
 [100000 102000]
 [106500  62500]
 [ 72000 111000]]
```

8.3 `salary.reshape(-1, 2)` # Lets NumPy work out one side

SAMPLE OUTPUT

```
[[ 69500  91500]
 [100000 102000]
 [106500  62500]
 [ 72000 111000]]
```

8.4 `grid.T` # Flips rows and columns

SAMPLE OUTPUT

```
[[ 69500  91500 100000]
 [    40     37     36]]
```

8.5 `grid.flatten()` # Turns a grid into one line

SAMPLE OUTPUT

```
[ 69500     40  91500     37 100000     36]
```

8.6 `grid.ravel()` # Flattens without copying data

SAMPLE OUTPUT

```
[ 69500     40  91500     37 100000     36]
```

8.7 `salary[:, np.newaxis].shape` # Adds an extra dimension

SAMPLE OUTPUT

```
(8, 1)
```

8.8 `np.expand_dims(salary, axis=0).shape` # Adds a dimension at a position

SAMPLE OUTPUT

```
(1, 8)
```

8.9 `np.squeeze(np.array([[1, 2, 3]]))` # Removes dimensions of size one

SAMPLE OUTPUT

```
[1 2 3]
```

8.10 `np.resize(salary, (2, 3))` # Forces a new shape, repeating values

SAMPLE OUTPUT

```
[[ 69500  91500 100000]
 [102000 106500  62500]]
```

9. Combining & Splitting

11 functions

Join arrays or break them apart.

9.1 `np.concatenate([salary[:3], salary[5:]])` # Joins arrays end to end

SAMPLE OUTPUT

```
[ 69500  91500 100000  62500  72000 111000]
```

9.2 `np.vstack([salary[:4], age[:4]])` # Stacks arrays as rows

SAMPLE OUTPUT

```
[[ 69500  91500 100000 102000]
 [    40     37     36     41]]
```

9.3 `np.hstack([salary[:3], age[:3]])` # Joins arrays side by side

SAMPLE OUTPUT

```
[ 69500  91500 100000    40    37    36]
```

9.4 `np.column_stack([salary[:4], age[:4]])` # Pairs arrays into columns

SAMPLE OUTPUT

```
[[ 69500    40]
 [ 91500    37]
 [100000    36]
 [102000    41]]
```

9.5 `np.split(salary, 2)` # Splits into equal parts

SAMPLE OUTPUT

```
[array([ 69500,  91500, 100000, 102000]), array([106500,  62500,  72000, 111000])]
```

9.6 `np.array_split(salary, 3)` # Splits into uneven parts

SAMPLE OUTPUT

```
[array([ 69500,  91500, 100000]), array([102000, 106500,  62500]), array([ 72000, 111000])]
```

9.7 `np.append(salary[:4], 88000)` # Adds a value to the end

SAMPLE OUTPUT

```
[ 69500  91500 100000 102000  88000]
```

9.8 `np.insert(salary[:4], 1, 75000)` # Inserts a value at a position

SAMPLE OUTPUT

```
[ 69500  75000  91500 100000 102000]
```

9.9 `np.delete(salary, 0)` # Removes an item by position

SAMPLE OUTPUT

```
[ 91500 100000 102000 106500  62500  72000 111000]
```

9.10 `np.tile(np.array([1, 2]), 3)` # Repeats a whole array

SAMPLE OUTPUT

```
[1 2 1 2 1 2]
```

9.11 `np.repeat(np.array([1, 2]), 3)` # Repeats each item

SAMPLE OUTPUT

```
[1 1 1 2 2 2]
```

10. Copying & Modifying

8 functions

Change arrays without breaking originals.

10.1 `backup = salary.copy()` # Makes a safe independent copy

SAMPLE OUTPUT

```
[ 69500  91500 100000 102000 106500  62500  72000 111000]
```

10.2 `view = salary[:3]` # A slice shares the same memory

SAMPLE OUTPUT

```
[ 69500  91500 100000]
```

10.3 `np.clip(salary, 70000, 105000)` # Limits values to a range

SAMPLE OUTPUT

```
[ 70000  91500 100000 102000 105000  70000  72000 105000]
```

10.4 `salary.fill(0)` # Replaces every item with one value

SAMPLE OUTPUT

```
[0 0 0 0 0 0 0 0]
```

10.5 `np.put(arr, [0, 1], [1000, 2000])` # Sets values at given positions

SAMPLE OUTPUT

```
[ 1000  2000 100000 102000 106500  62500  72000 111000]
```

10.6 `np.place(arr, arr > 100000, 100000)` # Caps values that break a rule

SAMPLE OUTPUT

```
[ 69500  91500 100000 100000 100000  62500  72000 100000]
```

10.7 `np.round(rating)` # Rounds to whole numbers

SAMPLE OUTPUT

```
[4. 5. 3. 3. 3. 3. 3. 5.]
```

10.8 `rating.astype(int)` # Cuts off the decimal part

SAMPLE OUTPUT

```
[3 4 3 3 3 3 2 4]
```

11. Handling Missing Values

9 functions

Work with NaN (empty) numbers.

11.1 `pay = np.array([69500, np.nan, 100000, 102000])` # NaN marks a missing number

SAMPLE OUTPUT

```
[ 69500.      nan 100000. 102000.]
```

11.2 `np.isnan(pay)` # Finds which items are missing

SAMPLE OUTPUT

```
[False  True False False]
```

11.3 `np.isnan(pay).sum()` # Counts the missing values

SAMPLE OUTPUT

```
1
```

11.4 `pay.mean()` # Any NaN makes the result NaN

SAMPLE OUTPUT

```
nan
```

11.5 `np.nanmean(pay)` # Average that ignores missing values

SAMPLE OUTPUT

```
90500.0
```

11.6 `np.nansum(pay)` # Total that ignores missing values

SAMPLE OUTPUT

```
271500.0
```

11.7 `pay[~np.isnan(pay)]` # Removes the missing values

SAMPLE OUTPUT

```
[ 69500. 100000. 102000.]
```

11.8 `np.nan_to_num(pay, nan=0)` # Replaces missing values with zero

SAMPLE OUTPUT

```
[ 69500.      0. 100000. 102000.]
```

11.9 `np.nanmax(pay), np.nanmin(pay)` # Highest and lowest, ignoring NaN

SAMPLE OUTPUT

```
(102000.0, 69500.0)
```

12. Random Numbers

8 functions

Make sample data and random picks.

```
12.1 | rng = np.random.default_rng(42)    # Creates a random generator
```

SAMPLE OUTPUT

```
Generator
```

```
12.2 | rng.integers(20, 60, size=5)      # Random whole numbers in a range
```

SAMPLE OUTPUT

```
[23 50 46 37 37]
```

```
12.3 | rng.random(4).round(3)           # Random decimals between 0 and 1
```

SAMPLE OUTPUT

```
[0.774 0.439 0.859 0.697]
```

```
12.4 | rng.choice(salary, size=3)        # Picks random items from an array
```

SAMPLE OUTPUT

```
[69500 72000 62500]
```

```
12.5 | rng.choice(salary, size=3, replace=False)  # Picks without repeating
```

SAMPLE OUTPUT

```
[111000 69500 62500]
```

```
12.6 | rng.normal(80000, 15000, 4).round(0)      # Random bell-curve numbers
```

SAMPLE OUTPUT

```
[84571. 64400. 91257. 94108.]
```

```
12.7 | rng.shuffle(arr)                     # Shuffles an array in place
```

SAMPLE OUTPUT

```
[102000 106500 100000 111000 72000 91500 62500 69500]
```

```
12.8 | rng.permutation(age)                # Returns a shuffled copy
```

SAMPLE OUTPUT

```
[41 24 36 38 45 37 59 40]
```

13. Linear Algebra

8 functions

Matrix maths used in data science.

13.1 `a @ b` # Multiplies two matrices

SAMPLE OUTPUT

```
[[19 22]
 [43 50]]
```

13.2 `np.dot(salary[:3], age[:3])` # Dot product of two arrays

SAMPLE OUTPUT

```
9765500
```

13.3 `a.T` # Transposes the matrix

SAMPLE OUTPUT

```
[[1 3]
 [2 4]]
```

13.4 `np.linalg.inv(a).round(2)` # Inverse of a matrix

SAMPLE OUTPUT

```
[[ -2.   1. ]
 [ 1.5 -0.5]]
```

13.5 `np.linalg.det(a).round(2)` # Determinant of a matrix

SAMPLE OUTPUT

```
-2.0
```

13.6 `np.trace(a)` # Sum of the diagonal

SAMPLE OUTPUT

```
5
```

13.7 `np.linalg.matrix_rank(a)` # Rank of the matrix

SAMPLE OUTPUT

```
2
```

13.8 `np.linalg.norm(salary[:3]).round(1)` # Length of a vector

SAMPLE OUTPUT

```
152323.7
```

14. Useful Utilities

12 functions

Small helpers that save time.

14.1 `np.array_equal(salary, salary.copy())` # Checks if two arrays match

SAMPLE OUTPUT

```
1
```

14.2 `np.set_printoptions(precision=2)` # Controls how numbers print

SAMPLE OUTPUT

```
[1.23 2.35]
```

14.3 `np.allclose([0.1 + 0.2], [0.3])` # Safe way to compare decimals

SAMPLE OUTPUT

```
1
```

14.4 `np.diff(salary)` # Difference between each pair

SAMPLE OUTPUT

```
[ 22000   8500   2000   4500 -44000   9500  39000]
```

14.5 `np.cumprod(np.array([1, 2, 3, 4]))` # Running product of items

SAMPLE OUTPUT

```
[ 1  2  6 24]
```

14.6 `np.sign(np.array([-5, 0, 3]))` # Shows if values are positive

SAMPLE OUTPUT

```
[-1  0  1]
```

14.7 `np.count_nonzero(salary > 90000)` # Counts items that match

SAMPLE OUTPUT

```
5
```

14.8 `np.savetxt("salary.csv", salary, delimiter=",")` # Saves an array to CSV

SAMPLE OUTPUT

```
6.9500000000000000e+04
```

14.9 `np.loadtxt("salary.csv", delimiter=",")` # Loads an array from CSV

SAMPLE OUTPUT

```
[ 69500.  91500. 100000. 102000. 106500.  62500.  72000. 111000.]
```

14.10 `np.save("salary.npy", salary)` # Saves in fast NumPy format

SAMPLE OUTPUT

```
[ 69500  91500 100000 102000 106500  62500  72000 111000]
```

14.11 `salary.mean().item()` # Converts to a plain Python number

SAMPLE OUTPUT

```
89375.0
```

14.12 `np.info(np.mean)` # Shows built-in help for a function

SAMPLE OUTPUT

Compute the arithmetic mean along the specified axis.
Returns the average of the array elements. The average is taken over the flattened array by default, otherwise over the specified axis.
`float64` intermediate and return values are used for integer inputs.